5장 토큰화, 6장 임베딩

5장 토큰화

자연어처리란

자연어 처리(Natural Language Processing, NLP)

- 컴퓨터가 인간의 언어를 이해하고 해석 및 생성하기 위한 기술

자연어 처리 모델 개발시 해결해야하는 문제

- 모호성(Ambiguity)
- 가변성(Variability)
- 구조(Structure)

→ 위와 같은 문제를 이해하고 구분할 수 있는 모델을 만드려면 우선 말뭉치(Corpus)를 일정한 단위인 토큰(Token)으로 나누어야 함

말뭉치

- 목적에 따라 구축되는 텍스트 데이터
- 일련의 단어의 가능성을 예측하는 알고리즘인 언어모델을 구축하고 평가하는 데 자주 사용됨

토큰

- 개별 단어나 문장부호와 같은 텍스트
- 말뭉치보다 더 작은 단위
- 토큰화 - 컴퓨터가 자연어를 이해할 수 있게 나누는 과정
- 토큰화에 토크나이저 사용

토크나이저 구축 방법

- 공백 분할
- 정규표현식 적용
- 어휘 사전(Vocabulary) 적용
  - 어휘 사전: 사전에 정의된 단어를 활용해 토크나이저 구축
  - 어휘 사전에 없는 단어나 토큰인 OOV(Out of Vocab) 문제를 고려해 토큰화하는 것이 중요
  - 큰 어휘 사전을 구축할 경우 학습 비용이 증가하고, 차원의 저주에 빠질 수 있음
- 머신러닝 활용

단어 토큰화

- 텍스트를 의미있는 단위인 단어로 분리하는 작업
- 단어 토큰화는 띄어쓰기, 문장 부호, 대소문자 등의 특정 구분자를 활용해 토큰화 수행
- 가장 일반적인 토큰화 방법

구현 방법

- 문자열 데이터 형태 - `split()` 메서드 사용

글자 토큰화

- 띄어쓰기뿐만 아니라 글자 단위로 문장을 나누는 방식

- 시퀀스 예측 작업에서 활용됨 ex: 다음 문제를 예측하는 언어 모델링

구현 방법

- 리스트 형태로 변환 - `list(sentence)`
- 한글의 경우 자소 단위로 나눠 자소 단위 토큰화 수행
    - `jamo` 라이브러리 활용

글자 토큰화의 장점

- 단어 토큰화의 단점이었던 접사와 문장 부호의 의미 학습 가능
- 비교적 작은 단어 사전을 구축할 수 있음
- 작은 크기의 단어 사전으로도 OOV 획기적으로 줄일 수 있음

글자 토큰화의 단점

- 개별 토큰은 아무런 의미가 없음 → 자연어 모델이 각 토큰의 의미를 조합해 결과를 도출해야 함
- 토큰 조합 방식을 사용해 개체명 인식 등을 구현할 경우 다의어나 동음이의어가 많은 도메인에서 구별하는 것이 어려움
- 모델 입력 시퀀스의 길이가 길어질수록 연산량 증가

형태소 토큰화

- 텍스트를 형태소 단위로 나누는 토큰화 방법으로 언어의 문법과 구조를 고려해 단어를 분리하고 이를 의미있는 단위로 분류하는 작업
- 한국어와 같이 교착어인 언어에서 중요하게 수행됨
    - 한국어는 대부분의 언어와 달리 각 단어가 띄어쓰기로 구분되지 않음

형태소 어휘 사전

- 각 단어의 형태소 정보를 포함하는 어휘 사전
- 단어가 어떤 형태소의 조합으로 이루어져 있는지에 대한 정보를 담고있음
- 각 형태소가 어떤 품사에 속하는지 + 해당 품사의 뜻 등의 정보도 함께 제공됨

품사(Part Of Speech, POS) 태깅

- 텍스트 데이터를 형태소 분석하여 각 형태소에 해당하는 품사를 태깅하는 작업
- 자연어 처리 분야에서 문맥 고려 가능해져 더욱 정확한 분석 가능해짐

KoNLPy

- 자바 언어 기반의 한국어 형태소 분석기로 자바 개발 키트(Java Development Kit, J DK)기반으로 개발됨
- 명사 추출, 형태소 분석, 품사 태깅 등의 기능 제공
- 텍스트 데이터를 전처리하고 분석하기 위한 다양한 도구와 함수 제공
- Okt(OpenKoreanText), 꼬꼬마(Kkma), 코모란(Komoran), 한나눔(Hannanum), 메캅(Mecab)' 등의 다양한 형태소 분석기를 지원

NLTK (Natural Language Toolkit)

- 토큰화, 형태소 분석, 구문 분석, 개체명 인식, 감성 분석 등과 같은 기능 제공
- 주로 영어 자연어 처리를 위해 개발됐지만, 네덜란드어, 프랑스어, 독일어 등과 같은 다양한 언어의 자연어 처리를 위한 데이터와 모델을 제공

spaCy

- 사이썬(Cython) 기반으로 개발된 자연어 처리를 위한 기능을 제공하는 오픈 소스 라이브러리
- NLTK는 학습 목적으로 자연어 처리에 대한 다양한 알고리즘과 예제를 제공하는 반면, SpaCy는 효율적인 처리 속도와 높은 정확도를 제공하는 것을 목표로 함

- 객체 지향적으로 구현됨

하위 단어 토큰화

- 형태소 단위의 토큰화 - 컴퓨터가 자연어를 인간이 이해하는 방식과 비슷하게 처리하기 위한 효과적인 방법
- 하나의 단어가 빈번하게 사용되는 하위 단어(Subword)의 조합으로 나누어 토큰화하는 방법
  - ex) Reinforcement: ‘Rein’ + ‘force’ + ‘ment’
- 단어의 길이를 줄일 수 있어 처리 속도가 빨라지고, OOV 문제, 신조어, 은어, 고유어 등으로 인한 문제 완화 가능
- 바이트 페어 인코딩, 워드피스, 유니그램 모델 등으로 구현 가능

바이트 페어 인코딩

- 텍스트 데이터에서 가장 빈번하게 등장하는 글자 쌍의 조합을 찾아 부호화하는 압축 알고리즘
- 연속된 글자 쌍이 더 이상 나타나지 않거나 정해진 어휘 사전 크기에 도달할 때까지 조합 탐지와 부호화를 반복하며 이 과정에서 자주 등장하는 단어는 하나의 토큰으로 토큰화되고, 덜 등장하는 단어는 여러 토큰의 조합으로 표현됨

6장 임베딩

텍스트 벡터화(Text Vectorization)

- 텍스트를 숫자로 변환하는 과정
- 원-핫 인코딩, 빈도 벡터화 등이 있음

워드 임베딩

- 단어의 의미를 학습해 표현하는 기법
- 단어를 고정된 길이의 실수 벡터로 표현하는 방법으로, 단어의 의미를 벡터 공간에서 다른 단어와의 상대적 위치로 표현해 단어 간의 관계 추론
- 동적 임베딩 - 고정된 임베딩을 학습하기 때문에 다의어나 문맥 정보를 다루기 어렵다는 단점이 있어 인공 신경망을 활용해 동적 임베딩 기법 사용
- 원-핫 인코딩, 빈도 벡터화의 벡터의 희소성, 차원의 저주 문제 해결 가능

언어 모델

- 입력된 문장으로 각 문장을 생성할 수 있는 확률을 계산하는 모델
- 문장 전체를 예측하는 대신 하나의 토큰 단위로 예측하는 방법인 자기회귀 언어 모델이 고안됨

자기회귀 언어 모델(Autoregressive Language Model)

- 입력된 문장들의 조건부 확률을 이용해 다음에 올 단어 예측

언어 모델에서 조건부 확률

- 이전 단어들의 시퀀스가 주어졌을 때, 다음 단어의 확률을 계산하는 것
- 이전에 등장한 모든 토큰의 정보를 고려하며, 문장의 문맥 정보를 파악해 다음 단어를 생성함

수식 6.1 언어 모델의 조건부 확률

$$P(w_t \mid w_1, w_2, \cdots, w_{t-1}) = \frac{P(w_1, w_2, \cdots, w_t)}{P(w_1, w_2, \cdots, w_{t-1})}$$

- 조건부 확률의 연쇄법칙을 적용한 수식

수식 6.2 조건부 확률의 연쇄법칙을 적용한 언어 모델의 조건부 확률

$$P(w_t|w_1, w_2, \cdots, w_{t-1}) = P(w_1)P(w_2|w_1)\cdots P(w_t|w_1, w_2, \cdots, w_{t-1})$$

통계적 언어 모델

- 언어의 통계적 구조를 이용해 문장이나 단어의 시퀀스를 생성하거나 분석함
- 시퀀스에 대한 확률 분포를 추정해 문장의 문맥을 파악해 다음에 등장할 단어의 확률 예측
- 마르코브 체인을 이용해 구현됨

수식 6.3 빈도 기반의 조건부 확률

$$P(\text{만나서}|\text{안녕하세요}) = \frac{P(\text{안녕하세요 만나서})}{P(\text{안녕하세요})} = \frac{700}{1000}$$

$$P(\text{반갑습니다}|\text{안녕하세요}) = \frac{P(\text{안녕하세요 반갑습니다})}{P(\text{안녕하세요})} = \frac{100}{1000}$$

- 대규모 자연어 데이터 처리하는데 효과적이며, 딥러닝 등의 인공지능 기술이 발전하면서 더욱 강력한 모델 구현할 수 있게 됨

N-gram

- 가장 기초적인 통계적 언어 모델
- 텍스트에서 N개의 연속된 단어 시퀀스를 하나의 단위로 취급해 특정 단어 시퀀스가 등장할 확률 추정
- N이 1: 유니그램(Unigram), N이 2: 바이그램(Bigram), N이 3: 트라이그램 (Trigram)



그림 6.3 N이 1, 2, 3인 N-gram

- N-gram 언어 모델은 모든 토큰을 사용하지 않고 N-1개의 토큰만을 고려해 확률 계산

N-gram 언어 모델에서 t번째 토큰의 조건부 확률을 계산하는 수식

수식 6.4 N-gram에서의 조건부 확률

$$P(w_t|w_{t-1}, w_{t-2}, \cdots, w_{t-N+1})$$

TF-IDF

- 텍스트 문서에서 특정 단어의 중요도를 계산하는 방법으로, 문서 내에서 단어의 중요도를 평가하는데 사용되는 통계적인 가중치 의미
- BoW(Bag-of-Words)에 가중치 부여하는 방법
  - BoW: 문서나 문장을 단어의 집합으로 표현하는 방법

단어 빈도(TermFrequency, TF)

- 문서 내에서 특정 단어의 빈도 수를 나타내는 값

수식 6.5 단어 빈도

$$TF(t,d) = count(t,d)$$

- 문서 내에서 단어가 등장한 빈도 수를 계산하며, 해당 단어의 상대적인 중요도를 측정하는데 사용됨
- TF 값이 높을수록 해당 단어가 특정 문서에서 중요한 역할을 한다고 생각 할수도 있지만, 단어 자체가 특정 문서 내에서 자주 사용되는 단어이므로 전문 용어나 관용어로 간주 할 수있음

문서 빈도(Document Frequency, DF)

- 한 단어가 얼마나 많은 문서에 나타나는지 의미
- 특정 단어가 많은 문서에 나타나면 문서 집합에서 단어가 나타나는 횟수 계산

수식 6.6 문서 빈도

$$DF(t,D) = count(t \in d : d \in D)$$

- DF 값이 높으면 특정 단어가 많은 문서에서 등장 한다고 볼 수있음.
  그 단어는 일반적으로 널리 사용되며, 중요도가 낮을 수 있음

역문서 빈도(Inverse Document Frequency, IDF)

- 전체 문서 수를 문서 빈도로 나눈 다음 로그를 취한 값
- 문서 내에서 특정 단어가 얼마나 중요한지를 나타냄

수식 6.7 역문서 빈도

$$IDF(t,D) = \log\left(\frac{count(D)}{1 + DF(t,D)}\right)$$

- 문서에서 특정 단어의 등장 횟수가 적으면 IDF는 상대적으로 커짐

TF-IDF

- 문서 빈도와 역문서 빈도를 곱한 값

수식 6.8 TF–IDF

$$TF - IDF(t,d,D) = TF(t,d) \times IDF(t,d)$$

- 문서 내에서 단어가 자주 등장하지만, 전체 문서 내에서 해당 단어가 적게 등장하면 TF-IDF 값은 커짐
- 전체 문서에서 자주 등장할 확률이 높은 관사나 관용어 등의 가중치가 낮아짐